11 / 24 — Final Assessment: React Mastery (Whole Curriculum)

0. How to use this assessment

Covers: Phases 0–11: web and JavaScript foundations, React fundamentals, rendering and internals, state and hooks, architecture, design patterns, data fetching and state ecosystems, performance, testing, advanced rendering (SSR, hydration, Suspense, Server Components), production engineering, and senior/architect judgment.

Purpose. The earlier phase assessments check each phase in isolation. This one checks **integration**: can you answer the questions that define mastery, using knowledge from every phase together, and defend your answers with mechanisms instead of slogans? The core eight questions this curriculum was built to answer are:

1. What happens, step by step, when `setState` is called?
2. When is state preserved and when is it reset?
3. What is the difference between reconciliation and DOM mutation?
4. What is Fiber and what does it make possible?
5. How and why do stale closures happen, and how do you fix them?
6. What are effects for, when do they run, and when should you not use one?
7. How do SSR, hydration and React Server Components differ, and what goes wrong between server and client?
8. How do you choose between architectural options, and defend the choice?

Every part of this assessment feeds one or more of those eight. Section 9 (the *mastery map*) shows which question tests which skill.

Format. Closed notes for Parts 1 and 2. Open documentation (react.dev, MDN, library docs) is allowed for Parts 3–5, but not AI assistants and not the course answers. React 19.x, TypeScript 5.x, **StrictMode off** unless a question says otherwise. Allow 8–10 hours in total. Do Parts 1–2 in a single sitting; do the implementation tasks (Part 4) by *running* your code.

Honest self-grading rules

1. Write every answer before opening Section 6 (answer key). "I knew that" scores zero.
2. For predictions, write the complete log or DOM state. Partial answers receive partial credit per the key.
3. Code that was never executed is graded at most half. Tests must be written and passing.
4. Questions that touch version-sensitive behavior are marked ⚠. State the version you assume. The key distinguishes **public guarantees** from **implementation details**; claiming an implementation detail as a guarantee loses points.
5. When the key gives "key points", award points only for points you actually wrote, in your own words, with the mechanism (not just the term).

Categories and points (each category totals 100; assessment total 400)

Category	Where	Points
Explanation	Part 1: E1–E10 (10 × 10)	100
Reasoning	Part 2: R1–R6 predictions (6 × 10) + Part 5: R7–R9 architecture (14 + 13 + 13)	100
Debugging	Part 3: D1–D5 (5 × 20)	100
Implementation	Part 4: I1–I4 (4 × 25)	100

Part 1 — Explanation (100 points)

Answer each in 150–300 words. Include a short diagram or ordered list when it helps.

E1. (10) What happens when `setState` is called? Starting from a user click on a button whose handler calls `setCount(c => c + 1)`, describe everything that happens until the user sees the new count, and until passive effects have run. Cover: event dispatch, what `setCount` does synchronously (and what it does *not* do), batching, scheduling, the render phase (what is invoked, what is compared), the commit phase (what touches the DOM, in which order), when the browser paints, and when `useEffect` callbacks run. State what happens if the new state equals the old state.

E2. (10) State preservation. State is not stored in the DOM and not in the component function. Explain where it lives and what determines whether a piece of state survives a re-render. List the conditions under which state is **preserved**, **reset**, and **lost without you noticing**. Give three code patterns that unintentionally reset state and two ways to reset state on purpose.

E3. (10) Reconciliation versus DOM mutation. Distinguish the elements tree, the fiber tree and the DOM. Explain what React compares during reconciliation, what it does with the result, which phase performs DOM mutation, and why "React diffs the real DOM" and "the virtual DOM makes React fast" are each incorrect or misleading. Name one case where reconciliation does **not** produce minimal DOM work.

E4. (10) Fiber. Explain why the stack reconciler was replaced, what a fiber is (as a data structure and as a unit of work), how current and work-in-progress trees implement double buffering, and which capabilities this architecture enables. Separate what is a *public guarantee* from what is an *implementation detail*.

E5. (10) Stale closures. Define a stale closure in terms of render snapshots. Give three distinct React situations in which one arises, explain why React's behavior here is deliberate rather than a bug, and list four different fixes with the situations each is appropriate for. Include a ⚠ note on `useEffectEvent`.

E6. (10) Effects. State what effects are for, the exact timing difference between `useLayoutEffect` and `useEffect`, the order of effects and cleanups across a parent/child tree on mount and on update, what dependencies mean, what Strict Mode development does to effects and why, and five situations in which you should not use an effect.

E7. (10) SSR, hydration and Server Components. Explain: what server-side rendering produces; what hydration does and what it requires; what a hydration mismatch is and how React 18/19 recovers; what selective hydration is; what React Server Components are and why they are *not* SSR; what crosses the server/client boundary and what restrictions apply to each side. Name what is **not** hydrated.

E8. (10) Concurrent rendering. Explain urgent versus transition updates, what `useTransition` and `useDeferredValue` each do, how Suspense interacts with transitions, why concurrent rendering requires render purity, and three misconceptions (for example, "concurrent React is multi-threaded").

E9. (10) State and data architecture. Classify state into categories (server, URL, client domain, UI, form, derived, real-time) and name a default home for each. Explain how you decide between local state, lifted state, context, an external store, and a server-state cache. Explain why copying server data into a client store is usually an anti-pattern and when it is justified.

E10. (10) Performance reasoning. Explain why a component re-renders; the difference between render cost, commit cost and browser layout/paint cost; when `memo`, `useMemo`, `useCallback` help and when they hurt; how state colocation, virtualization, transitions and code splitting each address a different bottleneck; and the correct order of operations when someone says "the page is slow". ⚠ Mention what the React Compiler changes and what it does not.

Part 2 — Prediction tasks (60 points of the Reasoning category)

Assume React 19, `createRoot`, **no StrictMode**, real user events in a browser. `log` appends a string to an ordered array. Write the full output or state, and one sentence of justification.

R1. Snapshot semantics and updater functions (10)

```
function Counter() {
  const [n, setN] = useState(0);
  function onClick() {
    setN(n + 1);
    setN(n + 1);
    setN(x => x + 1);
    log(`handler ${n}`);
  }
  log(`render ${n}`);
  return <button onClick={onClick}>{n}</button>;
}
```

The page loads, and the button is clicked **once**. (a) What is the complete log? (b) What does the button show? (c) What changes if the third call is `setN(n + 1)` as well?

R2. Identity and state preservation (10)

```tsx
function Field({ label }: { label: string }) {
  const [v, setV] = useState('');
  return <label>{label}<input value={v} onChange={e => setV(e.target.value)} /></label>;
}

function A({ flip }: { flip: boolean }) {
  return flip ? <Field label="B" /> : <Field label="A" />;
}
function B({ flip }: { flip: boolean }) {
  return flip ? <Field key="B" label="B" /> : <Field key="A" label="A" />;
}
function C({ flip }: { flip: boolean }) {
  return flip ? <div><Field label="B" /></div> : <Field label="A" />;
}
```

For each of A, B, C: the user types `x` into the input while `flip` is false, then `flip` becomes true. What label and what input value are shown? Which DOM nodes are reused, and which are destroyed and recreated?

R3. Effect ordering (10)

```tsx
function Child({ id }: { id: number }) {
  useEffect(() => { log(`child effect ${id}`); return () => log(`child cleanup ${id}`); },
[id]);
  useLayoutEffect(() => { log(`child layout ${id}`); return () => log(`child layout cleanup
${id}`); }, [id]);
  log(`child render ${id}`);
  return null;
}

function Parent() {
  const [id, setId] = useState(1);
  useEffect(() => { log('parent effect'); }, []);
  useLayoutEffect(() => { log('parent layout'); }, []);
  log('parent render');
  return (<><button onClick={() => setId(2)}>go</button><Child id={id} /></>);
}
```

Write the complete log (a) on mount and (b) after one click on the button. Assume passive effects have been flushed by the time you read the log.

R4. A stale interval (10)

```
function Timer() {
  const [n, setN] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setN(n + 1), 1000);
    return () => clearInterval(id);
  }, []);
  return <p>{n}</p>;
}
```

(a) What does the paragraph show after 1 s, after 5 s, and after 60 s? (b) Why? (c) Give two different fixes and say what each changes about timing behavior.

R5. Memoization and referential identity (10)

```
const Child = memo(function Child({ onPick }: { onPick: () => void }) {
  log('child');
  return <button onClick={onPick}>pick</button>;
});

function Parent() {
  const [n, setN] = useState(0);
  const stable = useCallback(() => {}, []);
  return (
    <>
      <button onClick={() => setN(n + 1)}>{n}</button>
      <Child onPick={stable} />
      <Child onPick={() => {}} />
    </>
  );
}
```

(a) What is logged on mount? (b) What is logged after clicking the counter button once? (c) If `Parent` also passed `style={{ color: 'red' }}` to the first `Child` (and `Child` accepted it), what would change?

R6. Batching across asynchronous boundaries (10)

```
function Demo() {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  const [c, setC] = useState(0);
  const renders = useRef(0);
  renders.current += 1;
  log(`render #${renders.current} a=${a} b=${b} c=${c}`);

  async function onClick() {
    setA(1);
    setB(1);
    await new Promise(r => setTimeout(r, 0));
    setC(1);
    log('after await');
  }
  return <button onClick={onClick}>go</button>;
}
```

Write the complete log from page load to after the click's timeout has completed. (The ref mutation during render is a deliberate probe; with StrictMode off it is accurate.) Then state: what would be different in React 17 and why?

Part 3 — Debugging challenges (100 points)

For each: (1) identify every defect, (2) explain the mechanism that produces the symptom, (3) write a corrected version, (4) say how you would verify the fix (tool and observation). 20 points each.

D1. "Sometimes the wrong user is shown" (20)

```
function UserPanel({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);
  useEffect(() => {
    fetch(`/api/users/${userId}`).then(r => r.json()).then(setUser);
  }, [userId]);
  return user ? <Profile user={user} /> : <Spinner />;
}
```

Symptoms: when clicking quickly through a list, the panel sometimes shows a user other than the selected one; on slower networks it shows the previous user's profile for a moment after switching; failed requests leave the panel stuck.

D2. "Drafts jump between rows and vanish when I filter" (20)

```tsx
function List({ items }: { items: Item[] }) {
  const [filter, setFilter] = useState('');
  const Row = ({ item }: { item: Item }) => {
    const [draft, setDraft] = useState(item.name);
    return <li><input value={draft} onChange={e => setDraft(e.target.value)} /></li>;
  };
  return (
    <>
      <input value={filter} onChange={e => setFilter(e.target.value)} placeholder="filter" />
      <ul>
        {items.filter(i => i.name.includes(filter)).map((it, idx) => <Row key={idx} item={it}
/>)}
      </ul>
    </>
  );
}
```

Symptoms: typing in the filter box discards any edits made in row inputs; after you hoist `Row` out of `List` to "fix" that, row drafts remain but now attach to the wrong items when the filter changes.

D3. "The network tab shows an endless stream of requests" (20)

```tsx
function Results({ query }: { query: string }) {
  const options = { q: query, limit: 20 };
  const [data, setData] = useState<Item[]>([]);
  useEffect(() => {
    search(options).then(setData);
  }, [options]);
  return <ul>{data.map(d => <li key={d.id}>{d.title}</li>)}</ul>;
}
```

D4. "Works in dev, throws on the server, flickers on the client" (20)

```tsx
function CartBadge() {
  const [items] = useState<string[]>(() => JSON.parse(localStorage.getItem('cart') ?? '[]'));
  return <span>{items.length} items · {new Date().toLocaleDateString()}</span>;
}
```

The component is rendered by a streaming SSR framework. Symptoms: the server logs `ReferenceError: localStorage is not defined`; after you guard that with `typeof window !== 'undefined'`, the browser console reports a hydration mismatch for some users, and the badge flashes.

D5. "Typing in an unrelated search box lags the whole dashboard" (20)

```tsx
const Ctx = createContext<{ user: User; theme: string; ticks: number; setTheme: (t: string) =>
void } | null>(null);

function App() {
  const [user] = useState<User>(loadUser());
  const [theme, setTheme] = useState('light');
  const [ticks, setTicks] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setTicks(t => t + 1), 100);
    return () => clearInterval(id);
  }, []);
  return (
    <Ctx.Provider value={{ user, theme, ticks, setTheme }}>
      <Dashboard />
    </Ctx.Provider>
  );
}
```

`Dashboard` contains 300 components, a search box with local state, and one small `<LiveClock />` that reads `ticks`. Profile data shows every component re-rendering ten times a second.

Part 4 — Implementation tasks (100 points)

Write TypeScript that compiles, **run it**, and include tests (Vitest + React Testing Library). 25 points each.

I1. `useInterval(callback, delay)`**.** A hook that always calls the *latest* `callback` without resetting the interval when the callback changes, supports `delay === null` to pause, cleans up on unmount, and is safe under StrictMode. Demonstrate with a counter that uses `setN(n + 1)` (not an updater) and with a test using fake timers.

I2. `createStore` **+** `useStore`**.** A tiny external store: `getState`, `setState(partial | updater)`, `subscribe`, and a React hook `useStore(selector)` built on `useSyncExternalStore` that re-renders a component only when its selected slice changes (by `Object.is`). Include a `useShallow`-style helper so object-returning selectors don't loop. Tests must show that unrelated updates do not re-render a subscriber.

I3. Accessible `Tabs`**.** A compound component (`Tabs`, `Tabs.List`, `Tabs.Tab`, `Tabs.Panel`) supporting controlled (`value` / `onValueChange`) and uncontrolled (`defaultValue`) usage, ARIA roles/relationships per the tabs pattern, roving `tabIndex`, and Arrow/Home/End keyboard navigation. Tests: roles and attributes, keyboard navigation, controlled mode.

I4. `useSearch`**.** A hook `useSearch(query, fetcher, delayMs)` returning `{ status, data, error }` with debounced requests, cancellation of superseded requests via `AbortController`, protection against out-of-order responses, an idle state for empty queries, and error handling. Tests with fake timers and a fetcher that responds out of order.

Part 5 — Architecture and judgment (40 points of the Reasoning category)

Answer in 400–700 words each. Assumptions you state count; ungrounded assertions don't.

R7. (14) State architecture. An analytics dashboard has 12 widgets, a global date-range and region filter that users must be able to share by link, REST data per widget with different freshness needs, three widgets fed by a WebSocket, a drill-down modal, three feature teams, and a requirement that typing in the filter bar never stalls the page. Specify where every category of state lives, the data fetching strategy, how updates reach widgets, how widgets avoid unnecessary re-renders, loading and error granularity, and what you would measure to validate the design.

R8. (13) Rendering strategy. A product has a public marketing site, a content-heavy catalog (SEO-critical, thousands of pages), and an authenticated application with real-time features. Choose rendering strategies per area (CSR, SSR, SSG/ISR, streaming, Server Components), define the client/server boundary, caching and personalization strategy, how auth is enforced, and the failure modes of your choices. Flag which parts depend on framework specifics you would verify against current docs.

R9. (13) Organization-scale decision. Six teams share one product. Currently one repository and one app; build time is 22 minutes; teams block each other on releases; the design system is copied between folders. Decide among: staying a monolith with stronger boundaries, a monorepo with packages, microfrontends, or multiple repositories. Write the decision as an ADR (context, options, decision, consequences, reversibility, dissent, review date) and a 90-day migration plan with fitness functions.

---

6. Answer key

> Attempt every question before reading this. Points are awarded per key point, in your own words, with the mechanism.

Part 1 key

E1 — What happens on `setState` (2 points per bullet)

1. *Event dispatch.* The browser dispatches the native click; React's root listener (React attaches listeners at the root container, not on each element) finds the handler through the fiber tree and calls it. Discrete user events carry high priority. The handler runs inside the *current render's closure*.
2. `setCount` *synchronously only enqueues.* It creates an update object on the hook's queue, marks the fiber (and ancestors up to the root) as having pending work in a lane, and schedules the root to be processed. It does **not** change `count` in the running closure and does **not** render synchronously in the middle of the handler. (An "eager" bailout may compare against the current state when the queue is empty: an implementation optimization.)
3. *Batching.* Updates made in the same event (and, since React 18, in promises, timeouts and native handlers within the same tick) are collected and processed together; the render happens after the

handler returns (for discrete events, flushed in a microtask or at the end of the event; the exact flush timing is an implementation detail, the public guarantee is "batched, after the handler").
4. *Render phase.* React re-invokes the components on the path to the work (re-calls the function; `useState` returns the new value after processing the queue, folding updater functions in order), producing new elements. Children are reconciled against the previous fibers by type and key; components whose props are the same references *and* have no pending work/context change can be skipped (bailout; `memo` extends skipping to cases where the parent rendered). The phase is pure, may be paused, restarted or discarded in concurrent rendering, and **does not touch the DOM**.
5. *Commit phase.* If anything changed: DOM mutations are applied (insert/update/delete), refs are detached/attached, the work-in-progress tree becomes current, layout effects run synchronously (cleanups of changed layout effects before mutations are applied, setups after), before the browser can paint. The browser then performs style, layout and paint. Passive effects (`useEffect`) run afterward (typically after paint; React may flush them earlier, for example before the next render or synchronously at the end of a commit caused by a discrete event: timing details are implementation; the guarantee is "after the commit, cleanup before the next run").
6. *Bail out.* If the new state is `Object.is` -equal to the old, React may render the component once more but skip rendering children and committing effects (documented); no DOM change.

E2 — State preservation (2 points per bullet)

1. State lives in React's internal fiber for that component *instance* (in a hooks list), not in the DOM or the function's local variables; identity of the instance is (position in parent, element type, key).
2. *Preserved:* same type at the same position (and same key) across re-renders even when props change; parent re-renders; hiding via CSS.
3. *Reset:* different type at the same position; different key; unmounted (conditionally removed); moved to a different parent/wrapper; the whole subtree remounted by an ancestor reset.
4. *Unintentional resets:* a component *defined inside* another component (new type each render); conditional wrapper elements (`cond ? <div><X/></div> : <X/>`); unstable or random keys (`key={Math.random()}` , `key={index}` on reorderable lists causes the *wrong* preservation); changing `key` to derived values that vary.
5. *Intentional resets:* give the component a `key` tied to the identity (e.g. `key={userId}`) so a new identity gets fresh state; lift state and reset explicitly; `<Activity>` (stable since React 19.2) can keep the state of hidden UI while it is hidden. *Lost without noticing:* uncontrolled DOM state (scroll, focus) is separate from React state; remounting loses focus and scroll.

E3 — Reconciliation vs DOM mutation (2 points per bullet)

1. Elements are plain objects describing UI for one render (`type` , `props` , `key`); fibers are React's internal instances/work units; DOM nodes are host objects.
2. Reconciliation compares the *new element tree* with the *previous fiber tree* using heuristics (different types ⇒ replace subtree; same type ⇒ update; keys match children in lists), producing flags (placement, update, deletion). It reads no DOM.
3. DOM mutation happens in the *commit phase*: React applies the computed effects (set changed attributes/text, insert, remove, move) to the DOM.
4. "React diffs the real DOM" is wrong (except hydration, which compares expectations against existing DOM). "VDOM makes React fast" is misleading: rendering costs JS work; the value of the

model is declarative UI and batched, minimal mutations, not that diffing is free. A render with no DOM change still costs.
5. *Non-minimal case:* changing an element's type (or key) recreates the entire subtree and loses state; list reorder with index keys updates props/text in place instead of moving nodes (and may misattribute state); a `display`-affecting parent change can force big layout even if DOM operations are few.

E4 — Fiber (2 points per bullet)

1. *Why:* the stack reconciler was recursive and synchronous: once started it could not be paused, so long renders blocked input and animation, and priorities could not be expressed.
2. *Fiber as structure:* a JS object per component instance/host node with type, props, state/hooks (`memoizedState`), effect flags, lanes, and `child` / `sibling` / `return` pointers forming a linked tree traversable iteratively, plus `alternate` pointing to its twin.
3. *Unit of work and double buffering:* the work loop processes one fiber at a time (begin work down, complete work up), can check whether to yield; the *current* tree (what's on screen) and the *work-in-progress* tree (being built) swap at commit, so an interrupted render can be discarded without having affected the screen.
4. *Enabled:* time slicing, update priorities (lanes), transitions, Suspense with retrying, error boundaries, selective hydration, offscreen work, abandoning stale renders.
5. *Guarantee vs detail:* public guarantees: rendering is pure and may be called more than once or discarded; effects run after commit; batching. Implementation details: field names, lane bit layout, exact scheduler heuristics, the order of internal passes (verify in source at your version; do not depend on them).

E5 — Stale closures (2 points per bullet)

1. *Definition:* every render creates new functions that close over *that render's* props and state (snapshots). A function that outlives its render (timers, subscriptions, async continuations, memoized callbacks) keeps seeing that snapshot.
2. *Situations:* `setInterval` / `setTimeout` created in an effect with `[]` deps; async callbacks after `await` reading state; event handlers or subscriptions registered once; memoized `useCallback` with stale deps; passing a callback to a non-React library once.
3. *Deliberate:* snapshots give each render a consistent view (essential for concurrent rendering and for reasoning: "state doesn't change mid-render or mid-handler"); mutable reads would make renders order-dependent.
4. *Fixes:* (a) functional updater `setN(n => n + 1)` when you only need the previous state to write; (b) correct dependencies so the effect re-synchronizes (with cleanup) when the value changes; (c) read from a ref that holds the latest value (updated in an effect, not during render) for callbacks that must not reset a subscription; (d) `useReducer`/store whose `dispatch` is stable; (e) restructure: move the logic into an event handler or derive during render.
5. `useEffectEvent` (stable API since React 19.2; verified on react.dev, 2026-10-22): for callbacks called only from Effects (or other Effect Events) that should always see the latest props/state without being dependencies. Restrictions: call it at the top level of a component or custom Hook; never call it during render or pass it to other components or Hooks; its returned function deliberately has **no stable identity** and must not appear in a dependency array (`eslint-plugin-react-hooks` enforces this). Don't use it to hide dependencies that should re-trigger synchronization.

E6 — Effects (2 points per bullet)

1. *Purpose:* synchronize with systems outside React (network subscriptions, DOM APIs, timers, third-party widgets, analytics). Not for deriving state, responding to events, or transforming data for rendering.
2. *Timing:* `useLayoutEffect` runs synchronously after DOM mutations and before the browser paints (use to measure layout and adjust before paint; blocks paint). `useEffect` runs after commit, usually after paint (don't assume exact timing), and is for non-visual synchronization.
3. *Order:* on mount: children's effects before parents'; all layout effects run before any passive effects in a commit. On update: for effects whose deps changed, cleanups run before the new setups (for layout: cleanup during mutation, setup during layout phase; for passive: all cleanups before all setups across components). Unmount: cleanups run.
4. *Dependencies:* the values the effect reads from render scope; React compares with `Object.is` after each render; effect re-synchronizes when they change. They are not a performance knob. Lint (`exhaustive-deps`) enforces truthfulness.
5. *Strict Mode (dev only):* mounts, simulates unmount (cleanups), and mounts again once to reveal missing cleanup/idempotence; it also double-invokes render functions to reveal impurity. Production runs once.
6. *Don't use an effect for:* derived state (compute in render), resetting state on prop change (use `key`), responding to a click (put it in the handler), fetching without cancellation (use a data library/framework loader), chaining state updates, notifying parents (call in handler). Also: effects don't run on the server.

E7 — SSR, hydration, RSC (2 points per bullet)

1. *SSR* renders components to HTML on the server (streamable, with Suspense boundaries flushing as they resolve) so users see content before JS loads; it runs the render phase, not effects or refs.
2. *Hydration* (`hydrateRoot`) reuses the existing DOM: React renders on the client and expects output to match the server HTML, attaches event handlers, and creates the fiber tree; the first client render must be identical to the server output.
3. *Mismatch:* differing text/markup (time, locale, random ids, `window` checks, extension-modified DOM). React 18+ logs the error (more descriptive in 19) and recovers by client-rendering the affected Suspense boundary (or root) instead of silently patching; this costs performance and may flash. Attribute mismatches are not guaranteed to be patched. `suppressHydrationWarning` is a shallow escape hatch for unavoidable text differences.
4. *Selective hydration:* Suspense boundaries hydrate independently; React can hydrate other parts first, prioritizes the boundary the user interacts with, and does not block the whole page on a slow chunk; discrete events during hydration may be replayed after hydration ⚠ implementation detail.
5. *RSC:* components that run only on the server (or at build time) and send a serialized description of their output (including references to client components) to the client; they are not hydrated, cannot use state/effects/browser APIs, can access server resources directly, and their props passed to client components must be serializable. SSR renders *client and server components* to HTML for first paint; RSC is about *where components execute and what code ships*. They compose: frameworks do both. ⚠ bundler/framework integration varies.
6. *Not hydrated:* Server Components (no client code), static HTML outside client boundaries in RSC frameworks, content in not-yet-hydrated Suspense boundaries until its code arrives.

E8 — Concurrent rendering (2 points per bullet)

1. Updates have priorities (lanes): urgent (typing, clicking) vs transition (non-urgent); React can interrupt an in-progress non-urgent render to handle urgent input, then restart.
2. `useTransition` / `startTransition` marks updates as non-urgent and provides `isPending`; `useDeferredValue` gives a lagging copy of a value that updates in a lower-priority render. Neither makes work cheaper; they make it interruptible and lower priority.
3. Suspense: a component that suspends (promise via `use` /library) shows the nearest fallback; within a transition, React prefers keeping the already-visible UI instead of replacing it with a fallback for content that was revealed.
4. Purity is required because render may run multiple times, be paused or be discarded; side effects in render would be duplicated or leak.
5. *Misconceptions:* it is not multi-threaded (still one main thread; it time-slices), not automatic speedup, not required to use (`createRoot` enables it but only transitions/deferred/Suspense change behavior), and not a license for impure render.

E9 — State and data architecture (2 points per bullet)

1. Categories and homes: server state → query cache / framework loaders / Server Components; URL state → router (shareable filters, selection); client domain state → feature-scoped store/reducer; UI state → local state; form state → form library / Actions; derived → computed; real-time → external store via `useSyncExternalStore`.
2. Decision ladder: start local; lift to the nearest common ancestor; use context for low-frequency dependency injection (theme, auth, DI), not for high-frequency state; external store for cross-cutting client state needing selective subscription; cache for server data.
3. Server state is a cache of remote truth with staleness, invalidation, deduplication, retries: duplicating it into a client store means rebuilding these badly and creating two sources of truth.
4. *When justified:* a local editable draft derived from server data (separate lifecycle), offline-first with a sync engine, or when the client store is the system of record temporarily (optimistic queue).
5. Colocate state with its readers; reason with ownership ("who is the single owner?") and write decisions down (ADR).

E10 — Performance reasoning (2 points per bullet)

1. A component re-renders when its own state changes, its parent re-renders (unless memoized with unchanged props), or a context it reads changes. A render does not imply a DOM update.
2. Costs: render (JS: function calls, reconciliation), commit (DOM mutation, effects), browser (style, layout, paint, composite). Different tools: Profiler vs Performance panel.
3. `memo` / `useMemo` / `useCallback` help only when they prevent measurable work (expensive subtrees, stable identity for effect deps or memoized children); they cost comparison, memory, dependency maintenance and can hide bugs; blanket use is counterproductive. ⚠ The React Compiler can add memoization automatically for rule-compliant code but doesn't fix architecture (state placement, list size, network waterfalls); verify current docs.
4. Fixes by bottleneck: colocate state (reduce blast radius); virtualization (huge lists); transitions/deferred values (keep input responsive); code splitting (initial load); caching/prefetching (waterfalls); web workers (CPU work).
5. Process: measure first (reproduce, profile with production build and throttling), identify the bottleneck class, change one thing, re-measure; verify in field data (INP/LCP); stop when budgets are met.

Part 2 key

R1. (a) Log: `render 0` (mount); after the click: `handler 0`, then `render 2`. (b) The button shows `2`. Reason: the handler's closure has `n = 0`; the first two calls enqueue "set to 1"; the third enqueues an updater that applies to the queue's current result (1) giving 2; one batched render follows the handler. `n` in the handler stays 0 throughout. (c) With three `setN(n + 1)`, the result is 1 (all set to 1); log is `handler 0`, `render 1`. (Scoring: 3 + 3 + 4.)

R2. A: label shows `B`, input shows `x`. Same type (`Field`) at the same position and no key: React updates props and preserves state and the DOM nodes (the `<label>` text updates, the `<input>` retains its value and state `v='x'`). B: label `B`, input empty. Different `key` means a different identity: the old `Field` fiber and its DOM nodes are destroyed, a new one is created with `v=''`. C: label `B`, input empty: element type at that position changed from `Field` to `div`, so the old subtree is unmounted and the new one mounted; all DOM nodes recreated. Common mistake: believing a changed `label` prop resets state. (Scoring: ~3, 3, 4.)

R3. (a) Mount: `parent render`, `child render 1`, `child layout 1`, `parent layout`, `child effect 1`, `parent effect`. Reason: render parent then child; layout effects run child-first then parent, all before passive effects; passive effects child-first. (b) After click: `parent render`, `child render 2`, `child layout cleanup 1`, `child layout 2`, `child cleanup 1`, `child effect 2`. The parent's effects have `[]` deps and do not re-run. Layout cleanup happens during the mutation phase, layout setup in the layout phase, then passive cleanup before passive setup. (Scoring: 5 + 5.) ⚠ The relative order of "layout setup" vs "passive cleanup" follows the documented ordering (layout before passive); the exact internal pass is implementation.

R4. (a) After 1 s the paragraph shows `1`; after 5 s still `1`; after 60 s still `1`. (b) The effect ran once with `n = 0` in its closure; every tick calls `setN(0 + 1)`; after the first tick `n` is 1 and subsequent identical sets bail out (no further renders). The interval keeps running with the stale closure. (c) Fix 1: `setN(c => c + 1)`: counter increments every second; interval created once; no dependency on `n`. Fix 2: add `n` to deps (`[n]`): the effect cleans up and recreates the interval after each tick; counts correctly but resets the timer each time (drift, extra setup cost). Also acceptable: `useInterval` with a latest-callback ref. (Scoring: 3 + 3 + 4.)

R5. (a) Mount: `child`, `child` (both render once). (b) After one click: `child` once: the first `Child` receives the stable `useCallback` reference and equal props, so `memo` skips it; the second receives a new arrow function each render so `memo`'s shallow comparison fails and it re-renders. (c) With an inline `style={{ color: 'red' }}`, the first `Child` also re-renders every time (a new object each render): log after click would be `child`, `child`. Fix by hoisting the constant object or memoizing it. Takeaway: `memo` only helps when *all* props are referentially stable. (Scoring: 2 + 4 + 4.)

R6. Log: `render #1 a=0 b=0 c=0` (mount). After the click: `render #2 a=1 b=1 c=0` (`setA` and `setB` are batched into one render), then after the timeout: `render #3 a=1 b=1 c=1` and `after await` (relative order of the last render and the `after await` log: the log statement executes synchronously right after `setC(1)` returns, so `after await` is logged **before** the render, because the render happens after the continuation finishes). So the full log is: `render #1 ...`, `render #2 a=1 b=1 c=0`, `after await`, `render #3 a=1 b=1 c=1`. The ordering of `after await` vs `render #3` is the key subtlety: state updates schedule, they don't run. In React 17 (legacy root, no automatic batching outside React event handlers), `setA` / `setB` inside the handler before the await are still batched (inside React's event handler), but `setC`

after the await (outside React's batching) renders synchronously, so `render #3` appears *before* `after await`. (Run-tested: react 17 gives `render a=0…`, `render a=1 b=1 c=0`, `render a=1 b=1 c=1`, `after await`; react 19.3.1 with `createRoot` gives `render #1`, `render #2`, `after await`, `render #3`.) In 18+ with `createRoot` the batching is automatic. (Scoring: 6 + 2 + 2.)

Part 3 key

D1 (20). Defects (2 points each, then fixes/verification):

1. *Race condition / no cancellation:* responses can arrive out of order; the last response to resolve wins, not the last request. Effects with async work need a cleanup that invalidates stale responses.
2. *Stale data displayed during load:* `user` isn't cleared when `userId` changes, so previous user shows until the new response arrives.
3. *No error handling:* `r.ok` unchecked, no `.catch`; failures leave the spinner forever, and unhandled rejections.
4. *No unmount protection:* update after unmount is harmless in React 18+ (no warning), but the race remains.
5. *Data fetching in effects reinvents a cache:* every mount refetches; no dedupe/retry.

Corrected (effect version):

```
type State = { status: 'loading' } | { status: 'error'; error: unknown } | { status:
'success'; user: User };

function UserPanel({ userId }: { userId: string }) {
  const [state, setState] = useState<State>({ status: 'loading' });
  useEffect(() => {
    const ac = new AbortController();
    setState({ status: 'loading' });
    fetch(`/api/users/${userId}`, { signal: ac.signal })
      .then(r => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return r.json() as
Promise<User>; })
      .then(user => setState({ status: 'success', user }))
      .catch(error => { if (!ac.signal.aborted) setState({ status: 'error', error }); });
    return () => ac.abort();
  }, [userId]);
  if (state.status === 'loading') return <Spinner />;
  if (state.status === 'error') return <ErrorMessage />;
  return <Profile user={state.user} />;
}
```

Better: use TanStack Query (`useQuery({ queryKey: ['user', userId], queryFn: ({ signal }) => ... })`) or a framework loader/Suspense, plus `key={userId}` on `<Profile>` if it holds local state. *Verification:* DevTools Network throttling ("Slow 4G") and rapid switching; add a test with MSW delaying the first response so it resolves after the second and assert the UI shows the second user; check the Network panel for canceled requests.

D2 (20). Defects:

1. *Component defined inside a component* (`Row`): a new function identity each `List` render ⇒ new element type ⇒ every row unmounts/remounts on each `List` render (typing in the filter), discarding `draft` state. (Mechanism: reconciliation sees a different `type`.) 6 points.
2. *Index keys* on a filtered (non-stable) list: after hoisting, keys are positions, so when filtering removes earlier items, a row at position 0 now represents a different item but React reuses fiber 0 with its old `draft` state. 6 points.
3. *State initialized from props* (`useState(item.name)`) is not updated when `item` changes: "derived state" smell; decide whether the row is controlled by parent or has an independent draft keyed by item identity. 4 points.
4. Missing `<label>`/accessible name on inputs. 2 points. Fix: hoist `Row` to module scope, key by `item.id`, store drafts keyed by id if they should survive filtering (lift to parent: `Record<string,string>`), or let uncontrolled rows keep their own state under stable keys; add labels.

Verification: React DevTools Profiler/Components "highlight updates" to see remounting (the component tree flashing; the Row displayed in DevTools as an anonymous/new instance); a test that types into a row, changes the filter, and asserts draft preserved/attached to the correct item.

D3 (20). Root cause: `options` is a new object every render ⇒ dependency changes after every render ⇒ the effect re-runs after every render ⇒ `search` resolves ⇒ `setData(newArray)` (new reference) ⇒ re-render ⇒ new `options` ⇒ effect again: an infinite loop of requests (6). Also: no cancellation/race protection (3), no loading/error state (3), setting state with fresh arrays defeats the bailout (2). Fix: depend on primitives and build the object inside the effect:

```
useEffect(() => {
  const ac = new AbortController();
  search({ q: query, limit: 20 }, { signal: ac.signal }).then(setData).catch(() => {});
  return () => ac.abort();
}, [query]);
```

or move constants out of the component (`const LIMIT = 20`), or use a query library keyed by `['search', query]`. The `exhaustive-deps` lint rule warns that `options` changes on every render (a clue to follow, not to silence with a disable comment: 4 points for noting lint evidence). Verification: Network panel request count, Profiler commit count, test counting calls to the mock `search`.

D4 (20). Defects: (1) reading `localStorage` during render breaks on the server (no `localStorage`) (4); (2) guarding with `typeof window` makes the server and client first renders differ (items length 0 on server vs real count on client; date text differs by timezone/locale/time) ⇒ hydration mismatch, recoverable error, client re-render and flash (6); (3) `new Date()` in render is impure and varies between server render time and hydration time (3); (4) `toLocaleDateString()` with default locale differs between server locale and user locale (3). Fix: render a deterministic placeholder on both, then update after mount, or use `useSyncExternalStore` with `getServerSnapshot`:

```
const subscribe = (cb: () => void) => { addEventListener('storage', cb); return () =>
removeEventListener('storage', cb); };
const getSnapshot = () => localStorage.getItem('cart') ?? '[]';   // string snapshot: stable
for equal content
const getServerSnapshot = () => '[]';

function CartBadge({ locale, timeZone, today }: { locale: string; timeZone: string; today:
string }) {
  const raw = useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot);
  const items = useMemo(() => JSON.parse(raw) as string[], [raw]);
  return <span>{items.length} items · {today}</span>;   // `today` computed on the server and
passed as a prop (same string used on client)
}
```

(Same-string snapshots avoid new-object loops; for same-tab updates, dispatch a custom event, since `storage` events fire only for other tabs.) For the date, pass server-formatted value as a prop or render after mount; `suppressHydrationWarning` on the single `<time>` element is an acceptable narrow escape hatch if the text may legitimately differ. Verification: view-source vs hydrated DOM; run with throttled CPU and a different system timezone/locale; check that no hydration warnings appear in the console.

D5 (20). Root causes: (1) `ticks` lives in the root with a 100 ms interval ⇒ `App` re-renders ten times a second (5). (2) `App` re-render creates new elements for `<Dashboard />` ⇒ non-memoized `Dashboard` and its whole subtree re-render regardless of context use (5). (3) the context `value` is a new object each render, so *every consumer* re-renders on any change (4). (4) one context mixes fast-changing (`ticks`) and slow (`user` , `theme`) data (3). (5) state placed far above where it's used (colocation violation) (3). Fix: move `ticks` into `LiveClock` itself (colocate) or into an external store that `LiveClock` subscribes to; split contexts (user/theme) and memoize provider values (or use a store with selectors); keep `App` free of fast-changing state so `<Dashboard />` isn't re-created; optionally `memo` heavy children. Profile before and after:

```
function LiveClock() {
  const [ticks, setTicks] = useState(0);
  useEffect(() => { const id = setInterval(() => setTicks(t => t + 1), 100); return () =>
clearInterval(id); }, []);
  return <span>{ticks}</span>;
}
```

Verification: React DevTools Profiler (commit count and "why did this render"), flame graph showing only `LiveClock` committing; INP measurement on the search box with CPU throttling.

Part 4 key (reference solutions and grading)

I1 (25). Grading: latest callback semantics (6), interval not reset on callback change (5), `null` pause (4), cleanup (4), test quality incl. fake timers (6).

```tsx
import { useEffect, useRef } from 'react';

export function useInterval(callback: () => void, delay: number | null) {
  const saved = useRef(callback);
  useEffect(() => { saved.current = callback; });           // runs after each commit: always
the latest
  useEffect(() => {
    if (delay === null) return;
    const id = setInterval(() => saved.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}
```

Notes: the ref is updated in an effect, not during render (purity). `useEffectEvent` (stable since React 19.2) is the React-sanctioned alternative for the callback read; the react.dev `useEffectEvent` page shows exactly this `useInterval` use case. Test: render a component with `useInterval(() => setN(n + 1), 1000)`, advance fake timers 3 s with `act`, expect 3; rerender with `delay=null` and ensure no further ticks; unmount and ensure `clearInterval` (no ticks).

I2 (25). Grading: store semantics (5), `useSyncExternalStore` with stable `subscribe` and cached snapshot (7), selector re-render isolation (6), `useShallow` or equivalent (4), tests (3).

```tsx
import { useRef, useSyncExternalStore } from 'react';

export function createStore<T extends object>(initial: T) {
  let state = initial;
  const listeners = new Set<() => void>();
  const getState = () => state;
  const setState = (p: Partial<T> | ((s: T) => Partial<T>)) => {
    const patch = typeof p === 'function' ? p(state) : p;
    const next = { ...state, ...patch };
    if ((Object.keys(patch) as (keyof T)[]).every(k => Object.is(state[k], next[k]))) return;
    state = next;
    listeners.forEach(l => l());
  };
  const subscribe = (l: () => void) => { listeners.add(l); return () => { listeners.delete(l);
}; };

  function useStore<R>(selector: (s: T) => R): R {
    return useSyncExternalStore(subscribe, () => selector(state), () => selector(state));
  }
  return { getState, setState, subscribe, useStore };
}

export function useShallow<S, R extends object>(selector: (s: S) => R) {
  const prev = useRef<R | undefined>(undefined);
  return (s: S): R => {
    const next = selector(s);
    const p = prev.current;
    if (p && shallowEqual(p, next)) return p;
    prev.current = next;
    return next;
  };
}
function shallowEqual(a: object, b: object) {
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every(k => Object.is((a as any)[k], (b as any)[k]));
}
```

Tests: component A selects `s => s.a`, component B selects `s => s.b`; use render counters (Profiler `onRender` or counts in a parent callback), call `setState({ b: 1 })`, assert A not re-rendered; verify an object selector without `useShallow` triggers the documented development warning/loop (or avoid asserting on it), and with `useShallow` renders only when a field changes. Edge: the server snapshot argument for SSR; singleton stores on the server leak across requests (create per request).

I3 (25). Grading: compound structure and context (5), controlled/uncontrolled (5), ARIA relationships (5), roving tabindex & keyboard (6), tests (4).

```tsx
import { createContext, useContext, useId, useRef, useState, type ReactNode, type
KeyboardEvent } from 'react';

type Ctx = { value: string; select: (v: string) => void; base: string };
const TabsContext = createContext<Ctx | null>(null);
const useTabs = () => { const c = useContext(TabsContext); if (!c) throw new Error('Tabs.*
must be inside <Tabs>'); return c; };

type TabsProps = { value?: string; defaultValue?: string; onValueChange?: (v: string) => void;
children: ReactNode };
export function Tabs({ value, defaultValue = '', onValueChange, children }: TabsProps) {
  const [inner, setInner] = useState(defaultValue);
  const controlled = value !== undefined;
  const current = controlled ? value : inner;
  const base = useId();
  const select = (v: string) => { if (!controlled) setInner(v); onValueChange?.(v); };
  return <TabsContext.Provider value={{ value: current, select, base }}>{children}
</TabsContext.Provider>;
}

Tabs.List = function List({ children, label }: { children: ReactNode; label: string }) {
  const ref = useRef<HTMLDivElement>(null);
  function onKeyDown(e: KeyboardEvent<HTMLDivElement>) {
    const tabs = Array.from(ref.current!.querySelectorAll<HTMLElement>
('[role="tab"]:not([disabled])'));
    const i = tabs.indexOf(document.activeElement as HTMLElement);
    if (i < 0) return;
    const next = e.key === 'ArrowRight' ? (i + 1) % tabs.length
      : e.key === 'ArrowLeft' ? (i - 1 + tabs.length) % tabs.length
      : e.key === 'Home' ? 0 : e.key === 'End' ? tabs.length - 1 : -1;
    if (next < 0) return;
    e.preventDefault();
    tabs[next].focus();
    tabs[next].click();               // automatic activation
  }
  return <div role="tablist" aria-label={label} ref={ref} onKeyDown={onKeyDown}>{children}
</div>;
};

Tabs.Tab = function Tab({ value, children }: { value: string; children: ReactNode }) {
  const { value: cur, select, base } = useTabs();
  const selected = cur === value;
  return (
    <button role="tab" type="button" id={`${base}-tab-${value}`} aria-selected={selected}
      aria-controls={`${base}-panel-${value}`} tabIndex={selected ? 0 : -1} onClick={() =>
select(value)}>
```

```
      {children}
    </button>
  );
};

Tabs.Panel = function Panel({ value, children }: { value: string; children: ReactNode }) {
  const { value: cur, base } = useTabs();
  return (
    <div role="tabpanel" id={`${base}-panel-${value}`} aria-labelledby={`${base}-
tab-${value}`}
      hidden={cur !== value} tabIndex={0}>
      {cur === value ? children : null}
    </div>
  );
};
```

Notes: `useId` output may contain colons: fine for `id` and ARIA references but not for CSS selectors without escaping. Edge cases (bonus): no initial selection when `defaultValue` is empty (should default to the first tab: acceptable to document), disabled tabs, RTL (swap arrow direction), vertical orientation (`aria-orientation`, Up/Down keys). Tests: roles present; `aria-selected` and `tabIndex` reflect selection; ArrowRight moves focus and selection; Home/End; controlled mode calls `onValueChange` and does not change until the parent updates. Pattern reference: WAI-ARIA Authoring Practices "Tabs" (verify at w3.org/WAI/ARIA/apg).

I4 (25). Grading: debounce (4), abort of superseded requests (6), ignoring stale results (5), idle/empty (3), error handling (3), tests incl. out-of-order (4).

```tsx
import { useEffect, useState } from 'react';

type Result<T> = { status: 'idle' | 'loading' | 'success' | 'error'; data: T[]; error?:
unknown };
const IDLE = { status: 'idle', data: [] } as const;

export function useSearch<T>(
  query: string,
  fetcher: (q: string, signal: AbortSignal) => Promise<T[]>,   // must be stable (module scope
or useCallback)
  delayMs = 250,
): Result<T> {
  const q = query.trim();
  const [state, setState] = useState<Result<T>>(IDLE as Result<T>);

  useEffect(() => {
    if (!q) return;
    const ac = new AbortController();
    setState(s => ({ ...s, status: 'loading' }));                // keep previous data while
loading (stale-while-revalidate)
    const t = setTimeout(() => {
      fetcher(q, ac.signal).then(
        data => { if (!ac.signal.aborted) setState({ status: 'success', data }); },
        error => { if (!ac.signal.aborted) setState({ status: 'error', data: [], error }); },
      );
    }, delayMs);
    return () => { clearTimeout(t); ac.abort(); };               // superseded: cancel timer
and request, ignore result
  }, [q, fetcher, delayMs]);

  return q ? state : (IDLE as Result<T>);
}
```

Tests: with fake timers, type `a`, then `ab` before the debounce elapses: fetcher called once with `ab`; type `ab` then `abc` after both requests started and resolve `abc` first, then `ab`: final data is for `abc` (the `ab` promise resolves after abort and is ignored); empty query returns idle and doesn't call the fetcher; a rejecting fetcher yields `error`; abort does not surface as an error. Notes: `setState` in an effect here is acceptable because it synchronizes the hook's state with an async external process; a library like TanStack Query would replace this hook in production code.

Part 5 key

R7 (14) — State architecture (evaluation criteria and example answer). Award: state taxonomy placement (4), data fetching/update flow (3), render isolation (3), loading/error granularity (2), measurement plan (2). Example strong answer:

- *URL state:* date range, region, selected widget/drill-down id live in search params (single source of truth, shareable, back button works); parse/validate with a schema; filter bar writes to the URL via the router, using `useTransition`/deferred value so typing stays responsive while widgets update.
- *Server state:* each widget has a query keyed by `[widget, filters]` in TanStack Query with its own `staleTime`/`refetchInterval` per freshness need; prefetch on filter hover/commit; dedupe shared endpoints; `placeholderData: keepPreviousData` to avoid flicker.
- *Real-time:* WebSocket client owned by a feature module (outside React), conflates messages per frame, writes to an external store (`useSyncExternalStore`) or patches the query cache for widgets whose data model is snapshot+delta; per-widget subscriptions only.
- *UI state:* modal open, hover, local tab: local state; modal route as URL param if deep-linkable.
- *Derived:* totals/formatting computed in selectors (queries' `select`), not stored.
- *Isolation:* widgets are independent subtrees with their own Suspense/error boundaries (granular failure); filter state read via narrow hooks; no top-level context carrying fast-changing data.
- *Teams:* each widget module owns its query + view; shared contracts for filters and design system; lint boundaries.
- *Measure:* INP on filter typing (field and lab with 4× CPU throttle), commit durations per widget (Profiler), request counts/dedupe, long tasks, error rates per widget. Common weak answers: global Redux for everything; filters in component state (not shareable); one context for all; no failure isolation.

R8 (13) — Rendering strategy. Award: per-area strategy with reasons (5), client/server boundary and data flow (3), caching/personalization/auth (3), failure modes and verification caveats (2). Example: Marketing: SSG/ISR on a CDN (maximum cacheability, no server per request), minimal client JS (server components or static). Catalog: ISR/SSR with edge caching and tag-based invalidation, streaming for slow modules (reviews, recommendations) behind Suspense; personalization (price tier, stock) in deferred/streamed sections or client fetch to keep HTML cacheable; structured data and canonical URLs for SEO; images optimized for LCP. App: authenticated; SSR or CSR depending on first-load needs; server components for data-heavy read views to cut bundle size if the framework supports them, client components for interactive and real-time parts (WebSocket store); auth enforced in middleware/BFF and again at data access (never only in the UI; server functions are public endpoints and must authorize); cookies HttpOnly; CSRF protection for mutations. Boundary rules: only serializable props cross; client components kept as leaves; secrets only on the server. Failure modes: hydration mismatch from personalization or time; cache poisoning (personalized HTML cached publicly: vary/cache-control discipline); stale caches after content changes; streaming with proxies that buffer; server function exposure; vendor lock-in. ⚠ Flag as verify-in-docs: framework caching semantics, RSC bundler integration, streaming behavior on your host.

R9 (13) — Organization-scale decision (ADR). Award: criteria and evidence (3), options analysis (3), decision with reversibility (3), ADR quality (2), 90-day plan and fitness functions (2). Example decision: modular monolith in a monorepo with enforced boundaries and remote caching, not microfrontends. Evidence to gather: what is the real cause of release blocking (coupled build/test vs organizational coupling)? 22-minute builds are addressed by caching, affected-only builds, parallelization (not by splitting deployables). Plan: Days 0–30: dependency graph analysis; extract the design system into a package with a changesets-based release; introduce workspace packages per feature; add boundary lint and dependency-cruiser; CODEOWNERS; CI affected tests and remote cache. Days 30–60: independent feature flags and per-team release trains; contract tests for shared APIs; performance budgets per route. Days 60–90: evaluate remaining pain; if independent deploy is *still* the bottleneck for specific teams, pilot route-level separate apps behind a reverse proxy for one team (strangler) before any runtime-composition

microfrontends. Consequences: shared version of React/deps (good for consistency, bad for independent upgrades); a single deployable may remain coupled (mitigate by flags and trunk-based delivery). Reversibility: high (packages can be split into repos later). Dissent recorded: a team wanting full autonomy via microfrontends; review date in six months with metrics (lead time, build time, release collisions). Fitness functions: no import across feature internals, domain layer has no React imports, bundle budgets, build time threshold, ownership coverage 100%.

---

7. Scoring rubric

Category	Points	Pass threshold (≥70%)
Explanation (E1–E10)	100	70
Reasoning (R1–R6 + R7–R9)	100	70
Debugging (D1–D5)	100	70
Implementation (I1–I4)	100	70
Total	**400**	**320 (80%)**

Quality bands per answer

- *Full credit:* correct mechanism, precise vocabulary, distinguishes guarantee from implementation detail, notes edge cases and version-sensitive behavior, proposes verification.
- *Half credit:* correct conclusion, but mechanism missing or hand-waved; or correct mechanism with a wrong detail that changes the conclusion in an edge case.
- *Zero:* correct-sounding slogans without mechanism ("React diffs the virtual DOM so it's fast"), or incorrect conclusions.

Deductions

- –2 per claim of an implementation detail (function names, internal ordering, lane numbers) stated as a public guarantee.
- –3 for code that does not compile or was not run (Part 4).
- –2 for each unverified URL cited as authoritative.

---

8. Promotion criteria — "React Master" certification

You pass this curriculum when **all** of these hold:

1. **Score:** ≥ 80% overall (320/400) and ≥ 70% in each of Explanation, Reasoning, Debugging and Implementation. A learner who passes only the conceptual parts does not advance.
2. **Phase assessments:** every phase assessment (Phases 0–10) passed at its own threshold.
3. **Projects:** Project 10 — Capstone completed to its review checklist, and Project 11 — Broken app diagnosed and refactored with written root-cause analysis.
4. **Oral defense (self-administered or with a peer):** deliver, without notes, a 10-minute explanation of each of the eight core questions at the top of this document, each including a diagram and one

concrete failure mode. Then run one timed design from 23 — System design interviews with a peer scoring against its rubric (average ≥ 4 on requirements, state/data, and trade-offs; ≥ 3 elsewhere).

5. **Review artifacts:** produce one ADR and one technical design document (see 01, 02) for a real or realistic decision, reviewed by a peer.

If you did not pass

Weak category / question	Review
E1, E3, E4, R1, R6 (setState, reconciliation, Fiber, batching)	Phase 2, lessons 01–23 and scenarios 32–37
E2, R2, D2 (identity and state preservation)	Phase 2 / 06–07, Phase 1
E5, E6, R3, R4, D1, D3, I1, I4 (closures and effects)	Phase 0 / 06, Phase 3 / 04–07
E7, D4 (SSR, hydration, RSC)	Phase 2 / 25–27, 37, Phase 9
E8 (concurrency)	Phase 2 / 13–16, 23, Phase 7 / 11–13
E9, E10, R5, D5, R7 (state and performance)	Phase 3 / 17–20, Phase 4 / 15–17, Phase 7
I2 (external stores)	Phase 3 / 20, Phase 6 / 10–11
I3 (accessible components)	Phase 5 / 05, 07, 18, Phase 8 / 10, Phase 4 / 20
R7–R9 (architecture)	Phase 4, Phase 11 lessons 01–16
Interview/system design	23 — System design interviews

Retake the failed category after two weeks, with new variants of the failed questions (invent them from the lesson exercises: change the code, keep the mechanism).

---

9. Mastery map: the eight questions and where they are tested

Core question	Tested in	Primary lessons
1. What happens on `setState`?	E1, R1, R6, D3	Phase 2 / 01–03, 11, 20, 22, Phase 3 / 01–02
2. State preservation	E2, R2, D2	Phase 2 / 06–07, Phase 3 / 17–18
3. Reconciliation vs DOM mutation	E3, R2, R5	Phase 2 / 04–06, 20, Phase 0 / 04
4. Fiber	E4	Phase 2 / 08–15, 21 — Reading framework source
5. Stale closures	E5, R4, I1	Phase 0 / 06, Phase 3 / 05
6. Effects	E6, R3, D1, D3, I4	Phase 2 / 18–19, 28, Phase 3 / 04–09
7. SSR / hydration / RSC	E7, D4, R8	Phase 2 / 25–27, 37, Phase 9 / 01–05
8. Architecture choices	E9, E10, R7–R9	Phase 4, Phase 6 / 19, Phase 11

---

10. Final notes

- The assessment tests **reasoning from mechanisms**. If a question feels like it could be answered by recalling a blog post title, you are missing the mechanism; work out *why* from render snapshots,

identity, commit ordering and purity.

- Versions change. Re-run the ⚠ items against the then-current React documentation each year; the mechanisms that matter (purity, snapshots, identity, batching, ordering of render/commit/effects) have been stable across major versions, while APIs and defaults evolve (22 — Long-term evolution).
- Mastery is maintained by practice: read source (21), review code (17), mentor (18), debug production issues (19), and keep evaluating new features with evidence (20).